

Linux Commands & Implementation Guide

Expanded Edition - concise, systematic revision notes with group management, full permission control and additional Linux topics

How to use this guide: learn the purpose first, then the syntax, then run the example. Commands shown with `sudo` may require administrator privileges. Replace placeholders such as `<user>`, `<file>`, `<dir>`, `<host>` and `<IP>` with your own values. Commands marked 'if installed' may need installing first. Examples are mostly for Ubuntu/Debian; RHEL-family equivalents are noted where they differ.

Added in this edition: filesystem hierarchy, shell shortcuts and wildcards, links, user and **group management**, `sudo/su`, a much larger **permissions** chapter (numeric/symbolic modes, SUID/SGID/sticky, `umask`, ACLs, `chattr`, capabilities), logs and `journalctl`, `cron/at`, `tar/gzip`, extended networking and firewall, SSH keys and tunnels, LVM and mounting, `dnf/rpm/pacman`, Bash arrays, strings, `getopts`, `trap`, troubleshooting playbook, security basics and more.

Contents

1. Linux Fundamentals	3
2. Command Structure, Help & Shell Productivity	3
3. Wildcards, Quoting & Expansion	4
4. System Information	4
5. CPU & Hardware	5
6. Memory & Performance	5
7. File System & Navigation	6
8. File Creation, Viewing & Editing	7
9. nano & vi/vim Quick Implementation	7
10. Copy, Move & Delete	8
11. Links (Hard & Symbolic)	8
12. Disk & Storage	8
13. Search & Text Processing	9
14. Pipes & Redirection	11
15. Regular Expressions - Quick Sheet	11
16. rename - Bulk Filename Changes	11
17. Users	12
18. Group Management (Grouping Users)	13
19. sudo, su & Privileges	14
20. Permissions & Ownership	14
21. Processes & Jobs	16

22. Services & systemd	17
23. Logs & Journals	18
24. Scheduling Jobs (cron, at, timers)	18
25. Networking	19
26. SSH - Remote Login	20
27. SCP - Secure File Copy	20
28. rsync - Synchronization & Backup	21
29. Archives & Compression (tar, gzip, zip)	21
30. Package Management	22
31. Shell Scripting - Core Structure	23
32. Bash Variables & Environment	23
33. Bash Special Variables	24
34. Bash Input, Tests & Conditions	24
35. if / elif / else, AND / OR	25
36. Loops	25
37. Reading Files in Bash	25
38. Functions	26
39. Arrays, Strings & Arithmetic in Bash	26
40. Script Robustness & Error Handling	26
41. Date & Time	27
42. Kernel, Boot & Power	27
43. Security & Hardening Basics	27
44. Other Useful Tools (git, containers, misc)	28
45. Practical Mini-Implementations	28
46. Troubleshooting Playbook	29
47. High-Value Command Differences	30
48. Quick Revision Cheat Sheet	30
49. Safety & Good Practice	31

1. Linux Fundamentals

- Linux: open-source OS ecosystem; the kernel is the core that manages CPU, memory, I/O, processes and files.
- CLI: command-line interface where commands are typed. Bash is the most common Linux shell.
- Shell: interprets commands, performs I/O redirection, manages environment variables and runs scripts (bash, sh, zsh, dash, fish).
- User space: applications with limited hardware access. Kernel space: privileged kernel access.
- Absolute path starts at /; relative path starts from the current working directory.
- Everything is a file: regular files, directories, devices (/dev), processes (/proc) and sockets.
- Prompt symbols: \$ = regular user, # = root, ~ = home, / = root of filesystem, . = current dir, .. = parent dir.
- Architecture: Hardware -> Kernel -> System Libraries -> Shell/Utilities -> User Applications.
- Distro families: Debian/Ubuntu (apt, .deb), RHEL/Fedora/CentOS/Rocky (dnf/yum, .rpm), Arch (pacman), SUSE (zypper).

Filesystem Hierarchy (FHS)

Command / Syntax	What it does	Example / Implementation
/	Root of the whole tree	
/bin, /sbin	Essential user / system binaries (often symlinks to /usr/bin, /usr/sbin)	
/boot	Kernel, initramfs, bootloader files	
/dev	Device files (sda, null, zero, tty)	
/etc	System-wide configuration files	/etc/passwd, /etc/ssh/sshd_config
/home	Home directories of normal users	
/root	Home directory of the root user	
/lib, /lib64	Shared libraries and kernel modules	
/media, /mnt	Mount points for removable media / temporary mounts	
/opt	Optional third-party software	
/proc, /sys	Virtual filesystems exposing kernel/process/hardware info	cat /proc/cpuinfo
/run	Runtime data (PID files, sockets) since boot	
/srv	Data served by the system (web, ftp)	
/tmp	Temporary files (cleared on reboot, sticky bit set)	
/usr	Installed programs, libraries, docs	/usr/bin, /usr/lib, /usr/share
/var	Variable data: logs, caches, spool, databases	/var/log, /var/spool

2. Command Structure, Help & Shell Productivity

- General form: command [options] [arguments]. Short options use one dash (-l), long options use two (--all). Short options can be combined (ls -lah).

Command / Syntax	What it does	Example / Implementation
command	Run a command	pwd
command --help	Quick help	ls --help
man command	Detailed manual page (q quits, /text searches)	man chmod
man -k keyword	Search manual pages by keyword (same as apropos)	man -k copy
man 5 passwd	Open a specific manual section (1 cmds, 5 files, 8 admin)	man 5 passwd
info command	GNU info documentation	info coreutils
whatis command	One-line description	whatis ls
apropos keyword	Find commands related to a keyword	apropos network
tldr command	Simplified community examples (if installed)	tldr tar
command --version	Show version	mysql --version
which command	Path of the executable	which python3
whereis command	Binary, source and manual locations	whereis bash

Command / Syntax	What it does	Example / Implementation
type command	Shows whether it is alias, builtin, function or file	type cd
history	Show previous commands	history tail
history -c	Clear this session's history	history -c
alias ll='ls -lah'	Create a shortcut (put in ~/.bashrc to persist)	alias ll='ls -lah'
unalias ll	Remove alias	unalias ll
clear	Clear the screen (or Ctrl+L)	clear
exit	Leave the shell / log out (or Ctrl+D)	exit

Keyboard Shortcuts (Bash)

Command / Syntax	What it does	Example / Implementation
Tab	Autocomplete command/file (double Tab lists options)	
Ctrl+C	Interrupt (kill) running command	
Ctrl+Z	Suspend command (resume with fg / bg)	
Ctrl+D	End of input / logout	
Ctrl+A / Ctrl+E	Jump to start / end of line	
Alt+B / Alt+F	Move back / forward one word	
Ctrl+U / Ctrl+K	Cut text before / after cursor	
Ctrl+W	Cut previous word	
Ctrl+Y	Paste what was cut	
Ctrl+R	Reverse search command history	
Ctrl+L	Clear screen	
!!	Repeat last command	sudo !!
!\$	Last argument of previous command	mkdir demo && cd !\$
!n	Run history entry number n	!105
Alt+.	Insert last argument of previous command	

3. Wildcards, Quoting & Expansion

- Globbing is done by the shell before the command runs.

Command / Syntax	What it does	Example / Implementation
*	Any characters (zero or more)	ls *.txt
?	Exactly one character	ls file?.txt
[abc] / [a-z] / [!0-9]	Character set / range / negated set	ls file[1-3].txt
{a,b,c}	Brace expansion (not globbing)	cp file.{txt,bak}
{1..10}	Sequence expansion	mkdir dir{1..5}
~	Home directory expansion	ls ~/Documents
\$VAR / \${VAR}	Variable expansion	echo "\${USER}_backup"
\$(cmd)	Command substitution	echo "Today is \$(date)"
\$((a+b))	Arithmetic expansion	echo \$((5*4))
'single quotes'	Literal text, no expansion	echo '\$HOME'
"double quotes"	Allows \$variable and \$(command) expansion	echo "Home: \$HOME"
\ (backslash)	Escape next character	echo \\$HOME / touch my\ file
;	Run commands one after another	cd /tmp; ls
&&	Run next only if previous succeeded	mkdir d && cd d
	Run next only if previous failed	cd d echo 'missing'
&	Run in background	sleep 60 &
#	Comment	# this is a comment

4. System Information

Command / Syntax	What it does	Example / Implementation
uname	Kernel/OS name	uname
uname -r	Kernel release	uname -r
uname -a	All system/kernel info	uname -a
uname -s	Kernel name only	uname -s
uname -v	Kernel build/version	uname -v
uname -m	Machine architecture	uname -m
uname -n	Hostname	uname -n
uname -o	Operating system name	uname -o
cat /etc/os-release	Distribution and version	cat /etc/os-release
lsb_release -a	Distribution info (if installed)	lsb_release -a
hostnamectl	Hostname, OS, kernel, architecture, chassis	hostnamectl
hostname	Show hostname	hostname
hostname -I	Show assigned IP addresses	hostname -I
hostname -f	Fully qualified domain name	hostname -f
sudo hostnamectl set-hostname <name>	Change hostname permanently	sudo hostnamectl set-hostname web-server01
cat /proc/version	Kernel version string	cat /proc/version
cat /etc/hostname	Stored hostname file	cat /etc/hostname
date	Current date/time	date
cal	Calendar of this month	cal / cal 2026
timedatectl	Time, timezone, NTP status	timedatectl
sudo timedatectl set-timezone <zone>	Change timezone	sudo timedatectl set-timezone Asia/Kolkata
uptime	Uptime, users, load averages	uptime
uptime -p	Pretty uptime	uptime -p
who -b	Last boot time	who -b
env	Environment variables	env head
printenv HOME	Print one variable	printenv PATH

5. CPU & Hardware

Command / Syntax	What it does	Example / Implementation
lscpu	CPU architecture, cores, model, virtualization	lscpu
cat /proc/cpuinfo	Detailed information for every CPU core	cat /proc/cpuinfo
nproc	Number of available processing units	nproc
sudo dmidecode -t system	System/BIOS/motherboard info	sudo dmidecode -t system
sudo dmidecode -t processor	Detailed processor info	sudo dmidecode -t processor
sudo dmidecode -t memory	Memory slot/configuration info	sudo dmidecode -t memory
sudo dmidecode -t bios	BIOS/UEFI info	sudo dmidecode -t bios
sudo lshw -short	Hardware summary (if installed)	sudo lshw -short
hwinfo --short	Short hardware summary (if installed)	hwinfo --short
lspci	List PCI devices	lspci
lspci -vv	Very detailed PCI information	lspci -vv
lsusb	List USB devices	lsusb
lsblk	Block devices	lsblk
lsmod	Loaded kernel modules	lsmod
ls SCSI	SCSI/SATA devices (if installed)	ls SCSI
sensors	Temperatures/fan speeds (lm-sensors)	sensors
inxi -Fx	Full system summary (if installed)	inxi -Fx

6. Memory & Performance

Command / Syntax	What it does	Example / Implementation
<code>free</code>	Memory and swap usage	<code>free</code>
<code>free -h</code>	Human-readable RAM/swap	<code>free -h</code>
<code>cat /proc/meminfo</code>	Detailed kernel memory statistics	<code>cat /proc/meminfo</code>
<code>vmstat -s</code>	Memory/swap/system statistics summary	<code>vmstat -s</code>
<code>vmstat 1 5</code>	Performance stats every 1 sec, 5 times	<code>vmstat 1 5</code>
<code>uptime</code>	Uptime, users and 1/5/15-minute load averages	<code>uptime</code>
<code>top</code>	Real-time process and CPU/RAM usage	<code>top</code>
<code>htop</code>	Interactive process monitor (if installed)	<code>htop</code>
<code>bttop</code>	Modern resource monitor (if installed)	<code>bttop</code>
<code>sar -u 1 5</code>	CPU activity, 5 samples (sysstat)	<code>sar -u 1 5</code>
<code>dstat</code>	Combined CPU, memory, disk, network (if installed)	<code>dstat</code>
<code>iostat -x 1 5</code>	CPU and detailed disk I/O stats	<code>iostat -x 1 5</code>
<code>mpstat -P ALL 1 5</code>	Per-CPU usage	<code>mpstat -P ALL 1 5</code>
<code>iotop</code>	Per-process disk I/O (root)	<code>sudo iotop</code>
<code>swapon --show</code>	Show swap devices	<code>swapon --show</code>

Reading top

- Keys inside top: q quit, k kill PID, M sort by memory, P sort by CPU, 1 show each CPU, u filter by user, h help.
- Load average: values near/below the number of CPU cores (nproc) are healthy; sustained higher values mean overload.

7. File System & Navigation

Command / Syntax	What it does	Example / Implementation
<code>pwd</code>	Print current working directory	<code>pwd</code>
<code>ls</code>	List files/directories	<code>ls</code>
<code>ls -l</code>	Long listing: permissions, owner, size, time	<code>ls -l</code>
<code>ls -a</code>	Include hidden files	<code>ls -a</code>
<code>ls -la / ls -lah</code>	Long listing, hidden files, human sizes	<code>ls -lah</code>
<code>ls -R</code>	Recursive listing	<code>ls -R</code>
<code>ls -r</code>	Reverse order	<code>ls -r</code>
<code>ls -S</code>	Sort by size (largest first)	<code>ls -S</code>
<code>ls -t</code>	Sort by modification time (newest first)	<code>ls -lt</code>
<code>ls -i</code>	Show inode numbers	<code>ls -i</code>
<code>ls -d */</code>	List only directories	<code>ls -d */</code>
<code>ls -ld <dir></code>	Show the directory itself, not contents	<code>ls -ld /tmp</code>
<code>ls ~</code>	List home directory	<code>ls ~</code>
<code>tree</code>	Directory tree view (if installed)	<code>tree -L 2</code>
<code>cd <dir></code>	Enter directory	<code>cd project</code>
<code>cd ..</code>	Parent directory	<code>cd ..</code>
<code>cd -</code>	Previous directory	<code>cd -</code>
<code>cd ~ / cd</code>	Home directory	<code>cd ~</code>
<code>cd /</code>	Filesystem root	<code>cd /</code>
<code>cd ~<user></code>	Another user's home (if accessible)	<code>cd ~cdac</code>
<code>pushd <dir> / popd</code>	Directory stack: save and return	<code>pushd /var/log ; popd</code>
<code>mkdir <dir></code>	Create directory	<code>mkdir project</code>
<code>mkdir -p a/b/c</code>	Create parent directories as needed	<code>mkdir -p project/src/test</code>
<code>mkdir -m 750 <dir></code>	Create with specific permissions	<code>mkdir -m 750 private</code>
<code>touch <file></code>	Create empty file / update timestamp	<code>touch notes.txt</code>
<code>file <file></code>	Detect file type	<code>file photo.jpg</code>
<code>stat <file></code>	Size, inode, permissions, timestamps	<code>stat notes.txt</code>

Command / Syntax	What it does	Example / Implementation
basename / dirname	Extract file name / directory part	basename /etc/passwd
realpath <file>	Absolute canonical path	realpath ../a.txt
readlink -f <link>	Resolve symlink target	readlink -f /usr/bin/python3

8. File Creation, Viewing & Editing

Command / Syntax	What it does	Example / Implementation
echo 'text'	Print text	echo 'Hello Linux'
echo 'text' > file	Create/overwrite file	echo 'Hello' > a.txt
echo 'text' >> file	Append to file	echo 'World' >> a.txt
echo -e 'a\nb'	Interpret escapes like \n and \t	echo -e 'Name\tAge'
printf '...\n' > file	Formatted file output	printf 'Name: %s\n' 'Aryan' > info.txt
cat file	Display file contents	cat notes.txt
cat -n file	Display with line numbers	cat -n notes.txt
cat f1 f2 > all	Join files	cat a.txt b.txt > ab.txt
cat > file	Type text, finish with Ctrl+D	cat > note.txt
tac file	Show lines in reverse order	tac log.txt
rev file	Reverse characters in each line	rev names.txt
nl file	Number lines	nl script.sh
less file	Page through a file (q quit, / search, G end, g start)	less /var/log/syslog
more file	Simple pager	more file.txt
head file / head -n 5 file	First 10 / first 5 lines	head -n 5 file.txt
tail file / tail -n 10 file	Last 10 / last 10 lines	tail -n 10 file.txt
tail -f logfile	Follow a growing log file	tail -f app.log
tail -F logfile	Follow, survives log rotation	tail -F /var/log/syslog
nano file	Simple terminal editor	nano notes.txt
vi / vim file	Modal advanced editor	vim script.sh
xdg-open file	Open with default desktop application	xdg-open report.pdf
truncate -s 0 file	Empty a file without deleting it	truncate -s 0 app.log

9. nano & vi/vim Quick Implementation

nano

- Ctrl+O save, Ctrl+X exit, Ctrl+K cut line, Ctrl+U paste, Ctrl+W search, Ctrl+\ search and replace.
- Ctrl+_ go to line, Ctrl+C show cursor position, Ctrl+A / Ctrl+E line start / end, Alt+U undo, Alt+E redo, Ctrl+G help.

vim - modes and insert

- Normal mode (default, for commands), Insert mode (typing), Visual mode (selecting), Command-line mode (:).
- i insert before cursor, a append after, I insert at line start, A append at line end, o new line below, O new line above, Esc back to Normal.

vim - navigation

- h/j/k/l left/down/up/right; w / b next / previous word; 0 line start; \$ line end; gg file start; G file end; 25G go to line 25; Ctrl+f / Ctrl+b page down / up; % jump to matching bracket.

vim - editing

- x delete char, dd delete line, 3dd delete 3 lines, dw delete word, D delete to end of line, cw change word, r replace char.
- yy copy line, yw copy word, p paste below, P paste above, u undo, Ctrl+r redo, . repeat last change.
- v select chars, V select lines, Ctrl+v block select; then d (delete), y (copy), > or < (indent).

vim - save, exit, search

- :w save, :q quit, :wq or ZZ save and quit, :q! quit without saving, :x save and exit.
- /pattern forward search, ?pattern backward, n next match, N previous match.
- :%s/old/new/g replace all in file; :%s/old/new/gc with confirmation; :s/old/new/ current line.
- :set number show line numbers; :set nonumber hide; :set hlsearch highlight matches; :noh clear highlight; :set paste paste mode.

- :e file open another file; :r file insert file contents; !cmd run a shell command; :sp / :vsp split window; Ctrl+w w switch window.

10. Copy, Move & Delete

Command / Syntax	What it does	Example / Implementation
cp source dest	Copy file	cp a.txt backup.txt
cp -r dir dest	Copy directory recursively	cp -r project backup/
cp -i source dest	Prompt before overwrite	cp -i a.txt b.txt
cp -n source dest	Never overwrite existing destination	cp -n a.txt backup/
cp -u source dest	Copy only if source is newer	cp -u a.txt backup/
cp -v source dest	Verbose (show what is copied)	cp -v a.txt backup/
cp -a source dest	Archive copy: recursive, preserves attributes and links	cp -a project/ backup/
cp -p source dest	Preserve mode, owner, timestamps	cp -p a.txt backup/
mv source dest	Move file/directory	mv a.txt docs/
mv old new	Rename	mv old.txt new.txt
mv -i source dest	Prompt before overwrite	mv -i report.txt backup/
mv -n source dest	Do not overwrite	mv -n a.txt docs/
rm file	Delete file	rm old.txt
rm -i file	Ask before deleting	rm -i old.txt
rm -r dir	Delete directory and contents	rm -r old_project/
rm -f file	Force removal, no prompts	rm -f temp.log
rm -rf dir	Force recursive delete - DANGEROUS, double-check path	rm -rf build/
rm -- -file	Delete a file whose name starts with -	rm -- -oddname
rmdir dir	Remove empty directory	rmdir emptydir
shred -u file	Overwrite then delete (secure delete)	shred -u secret.txt
install -m 755 src dst	Copy and set mode in one step	install -m 755 tool /usr/local/bin/
trash-put file	Move to Trash instead of deleting (trash-cli)	trash-put old.txt

11. Links (Hard & Symbolic)

- Hard link: another name for the same inode; survives deletion of the original name; cannot cross filesystems or link directories.
- Symbolic (soft) link: a small file pointing to a path; can cross filesystems; breaks if the target is removed.

Command / Syntax	What it does	Example / Implementation
ln source link	Create hard link	ln data.txt data_hard.txt
ln -s target link	Create symbolic link	ln -s /var/log/syslog ~/syslog
ln -sf target link	Replace existing symlink	ln -sf /opt/app-v2 /opt/app
ls -l	Symlinks show as l with -> target	ls -l ~/syslog
ls -i	Same inode number = hard links of same file	ls -i data.txt data_hard.txt
find . -xtype l	Find broken symlinks	find /etc -xtype l
unlink link	Remove a link	unlink ~/syslog

12. Disk & Storage

Command / Syntax	What it does	Example / Implementation
df	Filesystem disk usage	df
df -h	Human-readable usage	df -h
df -i	Inode usage	df -i
df -T	Show filesystem type	df -hT
du -sh <dir>	Total size of a directory	du -sh ~/Downloads
du -h --max-depth=1	Size of each sub-directory	du -h --max-depth=1 /var
du -sh * sort -h	Biggest items last	du -sh * sort -h
ncdu	Interactive disk usage browser (if installed)	ncdu /

Command / Syntax	What it does	Example / Implementation
lsblk	Disks, partitions, mount points (tree)	lsblk -f
sudo fdisk -l	Partition tables	sudo fdisk -l
sudo blkid	Partition UUIDs and filesystem types	sudo blkid
mount column -t	List mounted filesystems neatly	mount column -t
findmnt	Tree of mounts	findmnt
sudo mount /dev/sdb1 /mnt	Mount a partition	sudo mount /dev/sdb1 /mnt/usb
sudo mount -o ro <dev> <dir>	Mount read-only	sudo mount -o ro /dev/sdb1 /mnt
sudo umount /mnt	Unmount (path or device)	sudo umount /mnt/usb
sudo mount -a	Mount everything listed in /etc/fstab	sudo mount -a
cat /etc/fstab	Persistent mount table (device, mountpoint, type, options, dump, pass)	UUID=xxxx /data ext4 defaults 0 2
sudo fdisk /dev/sdb	Interactive partitioning (n new, d delete, p print, w write)	sudo fdisk /dev/sdb
sudo parted -l	List disks (GPT-friendly)	sudo parted -l
sudo mkfs.ext4 /dev/sdb1	Create ext4 filesystem (ERASES DATA)	sudo mkfs.ext4 /dev/sdb1
sudo mkfs.xfs /dev/sdb1	Create XFS filesystem	sudo mkfs.xfs /dev/sdb1
sudo fsck /dev/sdb1	Check/repair filesystem (unmounted only)	sudo fsck -f /dev/sdb1
sudo e2label <dev> <name>	Set ext label	sudo e2label /dev/sdb1 DATA
sudo mkswap /dev/sdb2	Create swap area	sudo mkswap /dev/sdb2 && sudo swapon /dev/sdb2
sudo resize2fs <dev>	Grow/shrink ext filesystem	sudo resize2fs /dev/sdb1
sudo smartctl -a /dev/sda	Disk health (smartmontools)	sudo smartctl -a /dev/sda
sudo dd if=<in> of=<out> bs=4M status=progress	Raw copy/clone - wrong 'of' destroys data	sudo dd if=disk.iso of=/dev/sdb bs=4M status=progress

LVM (Logical Volume Manager)

Command / Syntax	What it does	Example / Implementation
pvcreate /dev/sdb	Create physical volume	sudo pvcreate /dev/sdb
vgcreate vg0 /dev/sdb	Create volume group	sudo vgcreate vg0 /dev/sdb
lvcreate -L 10G -n data vg0	Create logical volume	sudo lvcreate -L 10G -n data vg0
lvextend -L +5G -r /dev/vg0/data	Grow LV and filesystem	sudo lvextend -L +5G -r /dev/vg0/data
pvs / vgs / lvs	Summaries of PV / VG / LV	sudo lvs

13. Search & Text Processing

grep

Command / Syntax	What it does	Example / Implementation
grep 'pattern' file	Search matching lines	grep 'ERROR' app.log
grep -n 'pattern' file	Show line numbers	grep -n 'ERROR' app.log
grep -i 'pattern' file	Case-insensitive	grep -i 'error' app.log
grep -v 'pattern' file	Show non-matching lines	grep -v '^#' config
grep -c 'pattern' file	Count matching lines	grep -c 'ERROR' app.log
grep -r 'pattern' dir	Recursive search	grep -r 'TODO' project/
grep -rin 'pattern' .	Recursive + case-insensitive + line numbers	grep -rin 'error' .
grep -w 'word' file	Whole-word match	grep -w 'main' code.c
grep -o 'pattern' file	Print only the matched text	grep -o '[0-9]\+' file.txt
grep -l 'pattern' *	List files that contain a match	grep -l 'password' *.conf
grep -E 'a b' file	Extended regex (egrep)	grep -E 'error fail' app.log
grep -A 2 -B 2 'x' file	Show 2 lines after / before match	grep -A2 -B2 'ERROR' app.log
grep -e p1 -e p2 file	Multiple patterns	grep -e ERROR -e WARN app.log
rg 'pattern' dir	Fast recursive search (if installed)	rg 'ERROR' .

find

Command / Syntax	What it does	Example / Implementation
find . -name 'file.txt'	Find by name	find . -name '*.log'
find . -iname '*.JPG'	Case-insensitive name	find . -iname '*.jpg'
find . -type f / -type d	Only files / only directories	find . -type d -name src
find . -type f -size +100M	Larger than 100 MB (-size -1M smaller)	find / -type f -size +100M
find . -type f -mtime -1	Modified within last day (+7 = older than 7 days)	find . -type f -mtime +7
find . -mmin -30	Modified in last 30 minutes	find . -mmin -30
find . -type f -empty	Empty files	find . -type f -empty
find . -user <u> / -group <g>	By owner / group	find /home -user cdac
find . -perm 777	By exact permission (-perm -4000 = SUID)	find / -perm -4000 2>/dev/null
find . -name '*.tmp' -delete	Delete matches	find . -name '*.tmp' -delete
find . -name '*.log' -exec rm {} \;	Run a command per match	find . -name '*.sh' -exec chmod +x {} \;
find . -name '*.txt' xargs wc -l	Pass results to another command	find . -name '*.txt' -print0 xargs -0 wc -l

locate / which

Command / Syntax	What it does	Example / Implementation
locate filename	Fast database search	locate report.txt
locate -i filename	Case-insensitive	locate -i report.txt
locate -n 10 filename	Limit results	locate -n 10 report.txt
sudo updatedb	Refresh locate database	sudo updatedb

counting, cutting, sorting

Command / Syntax	What it does	Example / Implementation
wc file	Lines, words, characters	wc report.txt
wc -l / -w / -m / -c / -L	Lines / words / chars / bytes / longest line	wc -l report.txt
cut -d':' -f1,3 file	Select delimited fields	cut -d':' -f1,3 /etc/passwd
cut -c1-5 file	Select characters 1-5	cut -c1-5 names.txt
sort file	Sort lines	sort names.txt
sort -n / -r / -u	Numeric / reverse / unique	sort -nr marks.txt
sort -k2 -t, file	Sort by 2nd field with comma delimiter	sort -t, -k2 -n data.csv
uniq	Remove adjacent duplicates (sort first)	sort names.txt uniq
uniq -c	Count duplicates	sort names.txt uniq -c
uniq -d	Show only duplicated lines	sort names.txt uniq -d
tr 'a-z' 'A-Z'	Translate characters	echo hello tr 'a-z' 'A-Z'
tr -d ' ' / tr -s ' '	Delete / squeeze characters	echo 'a b' tr -s ' '
sed 's/old/new/g' file	Replace text in stream	sed 's/error/ERROR/g' app.log
sed -i 's/old/new/g' file	Edit file in place (use -i.bak for backup)	sed -i.bak 's/foo/bar/g' a.txt
sed -n '5,10p' file	Print lines 5 to 10	sed -n '5,10p' app.log
sed '/^#/d' file	Delete matching lines (comments)	sed '/^#/d;/^\$/d' config
awk '{print \$1}' file	Print first field	awk '{print \$1}' data.txt
awk -F: '{print \$1,\$3}' file	Custom delimiter	awk -F: '{print \$1,\$3}' /etc/passwd
awk '\$3 > 100' file	Filter by condition	awk '\$3 > 100 {print \$1}' data.txt
awk '{s+=\$2} END{print s}' f	Sum a column	awk '{s+=\$2} END{print s}' sales.txt
paste f1 f2	Merge files side by side	paste names.txt marks.txt
join f1 f2	Join files on a common field	join a.txt b.txt
comm -12 f1 f2	Lines common to two sorted files	comm -12 a.txt b.txt
diff f1 f2	Show differences (-u unified, -r directories)	diff -u old.conf new.conf
cmp f1 f2	Byte-by-byte compare	cmp a.bin b.bin
column -t file	Align output in columns	mount column -t
xargs	Build commands from stdin	cat urls.txt xargs -n1 curl -O

14. Pipes & Redirection

Command / Syntax	What it does	Example / Implementation
>	Redirect stdout; overwrite	ls > files.txt
>>	Redirect stdout; append	date >> log.txt
<	Use file as stdin	sort < names.txt
	Send stdout of one command to another	ps aux grep python
2>	Redirect stderr	command 2> errors.txt
2>>	Append stderr	command 2>> errors.txt
2>&1	Send stderr to same place as stdout	command > all.log 2>&1
&>	Redirect stdout + stderr (Bash)	command &> all.log
> /dev/null	Discard output	command > /dev/null 2>&1
tee file	Show and save output at once	ls tee listing.txt
tee -a file	Show and append	echo hi tee -a log.txt
<< EOF	Here-document: multi-line input	cat << EOF > a.txt
<<< "text"	Here-string: feed a string as stdin	grep a <<< "banana"
<(cmd)	Process substitution	diff <(sort a.txt) <(sort b.txt)
cmd1 xargs cmd2	Convert stdin into arguments	ls *.bak xargs rm

- File descriptors: 0 = stdin, 1 = stdout, 2 = stderr.

15. Regular Expressions - Quick Sheet

Command / Syntax	What it does	Example / Implementation
^	Start of line/string	^ERROR
\$	End of line/string	done\$
.	Any single character	c.t
*	Zero or more of previous item	ab*
+	One or more of previous item (ERE)	ab+
?	Zero or one of previous item (ERE)	colour?r
{n,m}	Between n and m repetitions	a{2,4}
[abc]	One character from set	[aeiou]
[^abc]	One character not in set	[^0-9]
[a-z0-9]	Ranges	[A-Za-z]+
\d	Digit (PCRE: grep -P)	\d+
\w	Word character	\w+
\s	Whitespace	\s+
\b	Word boundary	\bc at\b
	OR (ERE)	cat dog
()	Group (ERE)	(ab)+
\.	Literal dot	\.log
[[[:digit:]] / [[[:alpha:]] / [[[:space:]]	POSIX classes	grep '[[[:digit:]]\{3\}' f

- Basic regex (grep) needs \+ \? \| \(\) escaped; use grep -E (extended) to write them unescaped. grep -P enables Perl syntax.

16. rename - Bulk Filename Changes

Command / Syntax	What it does	Example / Implementation
rename -n 's/_/-/g' *	PREVIEW changes first	Always preview with -n
rename 's/\.txt\$/ .md/' *.txt	Change extension	a.txt -> a.md
rename 's/^/new_/' *	Add prefix	a.txt -> new_a.txt
rename 's/ /_/g' *.txt	Replace spaces	my file.txt -> my_file.txt
rename 'y/a-z/A-Z/' *	Uppercase names	report.txt -> REPORT.TXT

Command / Syntax	What it does	Example / Implementation
<code>rename 'y/A-Z/a-z/' *</code>	Lowercase names	<code>REPORT.TXT -> report.txt</code>
<code>rename 's/draft/final/' *.docx</code>	Replace a word	<code>draft1.docx -> final1.docx</code>
<code>for f in *.txt; do mv "\$f" "\${f%.txt}.md"; done</code>	Bulk rename with plain Bash (no rename tool needed)	<code>for f in *.jpeg; do mv "\$f" "\${f%.jpeg}.jpg"; done</code>

17. Users

Viewing users

Command / Syntax	What it does	Example / Implementation
<code>whoami</code>	Current username	<code>whoami</code>
<code>echo \$USER</code>	Username from environment	<code>echo \$USER</code>
<code>who am i</code>	Your login session info	<code>who am i</code>
<code>users</code>	Names of logged-in users	<code>users</code>
<code>who</code>	Logged-in users with session info	<code>who</code>
<code>who -q / -b / -H / -u</code>	Count / last boot / headers / idle time	<code>who -H</code>
<code>w</code>	Who is logged in and what they run	<code>w</code>
<code>last</code>	Login/session history	<code>last -n 10</code>
<code>lastb</code>	Failed login attempts (root)	<code>sudo lastb</code>
<code>lastlog</code>	Last login of every user	<code>lastlog</code>
<code>id [user]</code>	UID, GID and groups	<code>id cdac</code>
<code>cat /etc/passwd</code>	Local user accounts (name:x:UID:GID:comment:home:shell)	<code>cat /etc/passwd</code>
<code>getent passwd</code>	Users from system account database (incl. LDAP)	<code>getent passwd cdac</code>
<code>sudo cat /etc/shadow</code>	Hashed passwords and aging info (root only)	<code>sudo cat /etc/shadow</code>
<code>awk -F: '\$3>=1000 {print \$1}' /etc/passwd</code>	List normal (human) users	<code>awk -F: '\$3>=1000{print \$1}' /etc/passwd</code>

Creating & modifying users

Command / Syntax	What it does	Example / Implementation
<code>sudo useradd -m <user></code>	Create user with home directory	<code>sudo useradd -m cdac</code>
<code>sudo useradd -m -s /bin/bash -G <grp> -c "Full Name" <user></code>	Create with shell, groups, comment	<code>sudo useradd -m -s /bin/bash -G developers -c "CDAC User" cdac</code>
<code>sudo useradd -u 1500 -d /home/x <user></code>	Custom UID and home path	<code>sudo useradd -u 1500 -m -d /home/x x</code>
<code>sudo useradd -r -s /usr/sbin/nologin <svc></code>	System (service) account with no login	<code>sudo useradd -r -s /usr/sbin/nologin appsvc</code>
<code>sudo adduser <user></code>	Debian/Ubuntu friendly interactive creation	<code>sudo adduser cdac</code>
<code>sudo passwd <user></code>	Set/change password	<code>sudo passwd cdac</code>
<code>passwd</code>	Change your own password	<code>passwd</code>
<code>sudo passwd -l <user> / -u <user></code>	Lock / unlock account	<code>sudo passwd -l cdac</code>
<code>sudo passwd -e <user></code>	Force password change at next login	<code>sudo passwd -e cdac</code>
<code>sudo passwd -S <user></code>	Password status	<code>sudo passwd -S cdac</code>
<code>sudo usermod -aG <group> <user></code>	Add to supplementary group (-a is essential)	<code>sudo usermod -aG developers cdac</code>
<code>sudo usermod -g <group> <user></code>	Change primary group	<code>sudo usermod -g staff cdac</code>
<code>sudo usermod -l <new> <old></code>	Rename login	<code>sudo usermod -l newname oldname</code>
<code>sudo usermod -d /new/home -m <user></code>	Change home directory and move files	<code>sudo usermod -d /home/n -m cdac</code>
<code>sudo usermod -s /bin/zsh <user></code>	Change login shell	<code>sudo usermod -s /bin/zsh cdac</code>
<code>sudo usermod -L <user> / -U <user></code>	Lock / unlock account	<code>sudo usermod -L cdac</code>
<code>sudo usermod -e 2026-12-31 <user></code>	Set account expiry date	<code>sudo usermod -e 2026-12-31 cdac</code>
<code>chsh -s /bin/bash</code>	Change own shell	<code>chsh -s /bin/bash</code>
<code>sudo chage -l <user></code>	Show password aging policy	<code>sudo chage -l cdac</code>
<code>sudo chage -M 90 -W 7 <user></code>	Max password age 90 days, warn 7 days	<code>sudo chage -M 90 -W 7 cdac</code>

Command / Syntax	What it does	Example / Implementation
<code>sudo gpasswd -d <user> <group></code>	Remove user from group	<code>sudo gpasswd -d cdac developers</code>
<code>sudo userdel <user></code>	Delete account, keep home	<code>sudo userdel cdac</code>
<code>sudo userdel -r <user></code>	Delete account and home	<code>sudo userdel -r cdac</code>
<code>sudo deluser --remove-home <user></code>	Debian-style delete with home	<code>sudo deluser --remove-home cdac</code>
<code>su - <user></code>	Switch user with their environment	<code>su - cdac</code>
<code>sudo pkill -KILL -u <user></code>	Kill all processes of a user before deletion	<code>sudo pkill -KILL -u cdac</code>

18. Group Management (Grouping Users)

- Every user has one primary group (GID in /etc/passwd) and any number of supplementary groups (listed in /etc/group).
- Groups let you give a set of users the same access to files, directories and commands.
- Files: /etc/group (name:x:GID:member1,member2) and /etc/gshadow (group passwords/admins).

View groups

Command / Syntax	What it does	Example / Implementation
<code>groups</code>	Groups of the current user	<code>groups</code>
<code>groups <user></code>	Groups of a given user	<code>groups cdac</code>
<code>id -nG <user></code>	Group names only	<code>id -nG cdac</code>
<code>id -gn <user></code>	Primary group name only	<code>id -gn cdac</code>
<code>cat /etc/group</code>	All groups and members	<code>cat /etc/group</code>
<code>getent group</code>	All groups (incl. LDAP/SSSD)	<code>getent group</code>
<code>getent group <group></code>	One group's entry and members	<code>getent group developers</code>
<code>grep '^developers:' /etc/group</code>	Quick member check	<code>grep '^developers:' /etc/group</code>
<code>lid -g <group> / members <group></code>	List members (if installed)	<code>members developers</code>

Create, rename, delete groups

Command / Syntax	What it does	Example / Implementation
<code>sudo groupadd <group></code>	Create a group	<code>sudo groupadd developers</code>
<code>sudo groupadd -g 2000 <group></code>	Create with a specific GID	<code>sudo groupadd -g 2000 devteam</code>
<code>sudo groupadd -r <group></code>	Create a system group	<code>sudo groupadd -r appsvc</code>
<code>sudo addgroup <group></code>	Debian/Ubuntu friendly version	<code>sudo addgroup testers</code>
<code>sudo groupmod -n <new> <old></code>	Rename group	<code>sudo groupmod -n devs developers</code>
<code>sudo groupmod -g 2500 <group></code>	Change GID	<code>sudo groupmod -g 2500 devs</code>
<code>sudo groupdel <group></code>	Delete group (cannot delete someone's primary group)	<code>sudo groupdel testers</code>

Add / remove members

Command / Syntax	What it does	Example / Implementation
<code>sudo usermod -aG <group> <user></code>	Append user to group (keep other groups)	<code>sudo usermod -aG developers cdac</code>
<code>sudo usermod -aG g1,g2 <user></code>	Add to several groups	<code>sudo usermod -aG developers,docker cdac</code>
<code>sudo usermod -G <groups> <user></code>	REPLACES all supplementary groups - dangerous without -a	<code>sudo usermod -G developers cdac</code>
<code>sudo gpasswd -a <user> <group></code>	Add one user to group	<code>sudo gpasswd -a cdac developers</code>
<code>sudo gpasswd -d <user> <group></code>	Remove user from group	<code>sudo gpasswd -d cdac developers</code>
<code>sudo gpasswd -M u1,u2,u3 <group></code>	Set the complete member list	<code>sudo gpasswd -M alice,bob dev</code>
<code>sudo gpasswd -A <user> <group></code>	Make user a group administrator	<code>sudo gpasswd -A alice developers</code>
<code>sudo adduser <user> <group></code>	Debian-style add to group	<code>sudo adduser cdac developers</code>
<code>sudo deluser <user> <group></code>	Debian-style remove from group	<code>sudo deluser cdac developers</code>
<code>sudo groupmems -g <group> -a <user></code>	Manage members (-l list, -d delete, -p purge)	<code>sudo groupmems -g developers -l</code>
<code>sudo useradd -m -g <primary> -G <extra> <user></code>	Create user with primary and extra groups	<code>sudo useradd -m -g staff -G developers,docker cdac</code>
<code>newgrp <group></code>	Start a shell with another active group	<code>newgrp developers</code>
<code>sg <group> -c 'command'</code>	Run one command as another group	<code>sg developers -c 'touch f.txt'</code>

- Group changes apply after the user logs out and in again (or run `newgrp / su - user`).

Group ownership of files

Command / Syntax	What it does	Example / Implementation
<code>sudo chgrp <group> file</code>	Change group of a file	<code>sudo chgrp developers report.txt</code>
<code>sudo chgrp -R <group> dir</code>	Recursive group change	<code>sudo chgrp -R devteam project/</code>
<code>sudo chown :<group> file</code>	Change only group using chown	<code>sudo chown :developers report.txt</code>
<code>find / -group <group> 2>/dev/null</code>	Find everything owned by a group	<code>find /home -group developers</code>

Recipe: shared team directory

```

sudo groupadd devteam          # 1. create group
sudo usermod -aG devteam alice # 2. add members
sudo usermod -aG devteam bob
sudo mkdir /srv/project        # 3. create shared folder
sudo chgrp devteam /srv/project # 4. give group ownership
sudo chmod 2770 /srv/project   # 5. rwx for owner+group, setgid so new files inherit group
sudo setfacl -d -m g:devteam:rwx /srv/project # 6. (optional) default ACL for new files
getent group devteam           # 7. verify

```

19. sudo, su & Privileges

Command / Syntax	What it does	Example / Implementation
<code>sudo <command></code>	Run command as root	<code>sudo apt update</code>
<code>sudo -u <user> <cmd></code>	Run as another user	<code>sudo -u postgres psql</code>
<code>sudo -i / sudo -s</code>	Root login shell / root shell	<code>sudo -i</code>
<code>sudo !!</code>	Repeat last command with sudo	<code>sudo !!</code>
<code>sudo -l</code>	List what you may run with sudo	<code>sudo -l</code>
<code>sudo -k</code>	Forget cached sudo credentials	<code>sudo -k</code>
<code>su - <user></code>	Switch user with login environment	<code>su - cdac</code>
<code>su</code>	Become root (needs root password)	<code>su -</code>
<code>sudo visudo</code>	Safely edit /etc/sudoers (syntax-checked)	<code>sudo visudo</code>
<code>/etc/sudoers.d/<file></code>	Drop-in sudo rules (edit with visudo -f)	<code>sudo visudo -f /etc/sudoers.d/devs</code>
<code>%group ALL=(ALL:ALL) ALL</code>	sudoers line: members of 'group' may run everything	<code>%sudo ALL=(ALL:ALL) ALL</code>
<code>user ALL=(ALL) NOPASSWD: /usr/bin/systemctl</code>	Allow one command without a password	<code>cdac ALL=(ALL) NOPASSWD: /usr/bin/systemctl</code>
<code>sudo usermod -aG sudo <user></code>	Give admin rights (Debian/Ubuntu group: sudo)	<code>sudo usermod -aG sudo cdac</code>
<code>sudo usermod -aG wheel <user></code>	Give admin rights (RHEL/Fedora group: wheel)	<code>sudo usermod -aG wheel cdac</code>

20. Permissions & Ownership

- Classes: u = user/owner, g = group, o = others, a = all.
- Bits: r = read (4), w = write (2), x = execute (1). Sum them: rwx = 7, rw- = 6, r-x = 5, r-- = 4, --x = 1, --- = 0.
- Meaning on files: r read content, w modify, x run as program. On directories: r list names, w create/delete/rename entries, x enter (cd) and access.
- Example: `-rwxr-xr--` = file; owner rwx (7), group r-x (5), others r-- (4) = 754. First character: - file, d directory, l symlink, b/c device.

Viewing

Command / Syntax	What it does	Example / Implementation
<code>ls -l file</code>	Check owner/group/permissions	<code>ls -l script.sh</code>
<code>ls -ld <dir></code>	Permissions of the directory itself	<code>ls -ld /tmp</code>
<code>stat -c '%a %U:%G %n' file</code>	Octal mode, owner, group	<code>stat -c '%a %U:%G %n' *</code>
<code>namei -l /path/to/file</code>	Permissions of every path component	<code>namei -l /var/www/html/index.html</code>

chmod - numeric (octal) mode

Command / Syntax	What it does	Example / Implementation
chmod 755 file	rw-r-xr-x: programs/directories	chmod 755 script.sh
chmod 750 file	Owner rwx, group r-x, others none	chmod 750 script.sh
chmod 644 file	rw-r--r--: normal documents	chmod 644 notes.txt
chmod 640 file	Owner rw, group r, others none	chmod 640 app.conf
chmod 600 file	Owner only (private keys!)	chmod 600 ~/.ssh/id_rsa
chmod 700 dir	Only the owner can enter	chmod 700 ~/private
chmod 666 file	rw for everyone (use carefully)	chmod 666 shared.txt
chmod 777 file	Full access for everyone (avoid)	chmod 777 /tmp/test
chmod 400 file	Read-only for owner	chmod 400 key.pem

chmod - symbolic mode

Command / Syntax	What it does	Example / Implementation
chmod a+x file	Add execute for everyone	chmod a+x script.sh
chmod u+x file	Execute for owner	chmod u+x script.sh
chmod g+w file	Add write for group	chmod g+w team.txt
chmod go-w file	Remove write from group and others	chmod go-w file.txt
chmod o-rwx file	Remove all access for others	chmod o-rwx secret.txt
chmod u=rwx,g=rx,o= file	Set exact permissions per class	chmod u=rwx,g=rx,o= file
chmod u=rx,g=r,o=x file	Another exact set	chmod u=rx,g=r,o=x file
chmod g=u file	Copy owner permissions to group	chmod g=u file
chmod +X -R dir	Add execute only to directories/already-executable files	chmod -R u+rwX,go+rX project/
chmod -R 755 dir	Recursive permission change	chmod -R 755 project/
chmod -c / -v file	Report changes made (-c) / verbose (-v)	chmod -v 644 a.txt
chmod --reference=src dst	Copy permissions from another file	chmod --reference=a.txt b.txt
find . -type f -exec chmod 644 {} +	Set files only (recursive)	find . -type f -exec chmod 644 {} +
find . -type d -exec chmod 755 {} +	Set directories only (recursive)	find . -type d -exec chmod 755 {} +

Ownership

Command / Syntax	What it does	Example / Implementation
sudo chown <user> file	Change owner	sudo chown cdac file.txt
sudo chown <user>:<group> file	Change owner and group	sudo chown cdac:DataAnalyst file.txt
sudo chown :<group> file	Change group only	sudo chown :developers file.txt
sudo chown -R <user>:<group> dir	Recursive ownership change	sudo chown -R cdac:cdac project/
sudo chown -h <user> link	Change the symlink itself, not target	sudo chown -h cdac mylink
sudo chown --reference=a b	Copy owner/group from another file	sudo chown --reference=a.txt b.txt
sudo chgrp <group> file	Change group	sudo chgrp developers report.txt
sudo chgrp -R <group> dir	Recursive group change	sudo chgrp -R devteam project/

Default permissions - umask

Command / Syntax	What it does	Example / Implementation
umask	Show current mask	umask
umask -S	Show mask symbolically	umask -S
umask 022	Files 644, directories 755 (typical default)	umask 022
umask 002	Files 664, directories 775 (group collaboration)	umask 002
umask 007	Files 660, directories 770	umask 007
umask 077	Files 600, directories 700 (private)	umask 077

- How umask works: new files start from 666, new directories from 777; subtract the mask (022 -> files 644, dirs 755). Put umask in ~/.bashrc to make it permanent.

Special permission bits

Command / Syntax	What it does	Example / Implementation
chmod u+s file / chmod 4755 file	SUID: run with the file owner's privileges	-rwsr-xr-x (e.g. /usr/bin/passwd)
chmod g+s dir / chmod 2775 dir	SGID: files run as group / new files in directory inherit its group	chmod 2775 /srv/shared
chmod +t dir / chmod 1777 dir	Sticky bit: only owner can delete their files in the directory	drwxrwxrwt (e.g. /tmp)
chmod u-s file	Remove SUID	chmod u-s file
find / -perm -4000 -type f 2>/dev/null	Audit all SUID programs	find / -perm -4000 -type f 2>/dev/null
find / -perm -2000 -type f 2>/dev/null	Audit SGID programs	find / -perm -2000 -type f 2>/dev/null
find / -perm -o+w -type f 2>/dev/null	Find world-writable files	find /var -perm -o+w -type f 2>/dev/null

- In ls -l: s/S in owner or group execute slot = SUID/SGID, t/T in others execute slot = sticky (capital means execute bit is not set). Four-digit modes: 1 = sticky, 2 = SGID, 4 = SUID (e.g. 2770, 4755).

ACL - fine-grained access control

Command / Syntax	What it does	Example / Implementation
getfacl file	View ACL entries	getfacl report.txt
setfacl -m u:<user>:rw file	Grant a specific user permission	setfacl -m u:cdac:rw report.txt
setfacl -m g:<group>:rx file	Grant a specific group permission	setfacl -m g:developers:rx report.txt
setfacl -R -m u:<user>:rwx dir	Recursive ACL	setfacl -R -m u:cdac:rwx project/
setfacl -d -m g:<group>:rwx dir	Default ACL inherited by new files	setfacl -d -m g:devteam:rwx /srv/project
setfacl -x u:<user> file	Remove one user's ACL entry	setfacl -x u:cdac report.txt
setfacl -b file	Remove all extended ACLs	setfacl -b report.txt
setfacl -k dir	Remove default ACLs	setfacl -k /srv/project
getfacl a setfacl --set-file=- b	Copy ACLs from one file to another	getfacl a.txt setfacl --set-file=- b.txt

- A '+' at the end of the ls -l permissions (-rw-rw-r--+) means ACLs are set.

File attributes (beyond permissions)

Command / Syntax	What it does	Example / Implementation
lsattr file	Show attributes	lsattr config.conf
sudo chattr +i file	Immutable: cannot be changed/deleted even by root	sudo chattr +i /etc/resolv.conf
sudo chattr -i file	Remove immutable flag	sudo chattr -i /etc/resolv.conf
sudo chattr +a file	Append-only (good for logs)	sudo chattr +a app.log

Capabilities

Command / Syntax	What it does	Example / Implementation
getcap -r / 2>/dev/null	List files with Linux capabilities	getcap /usr/bin/ping
sudo setcap cap_net_bind_service=+ep <bin>	Allow binding ports below 1024 without root	sudo setcap cap_net_bind_service=+ep ./server

21. Processes & Jobs

Command / Syntax	What it does	Example / Implementation
ps	Processes of current shell	ps
ps aux	All processes, detailed (BSD style)	ps aux
ps -ef	All processes, full format (UNIX style)	ps -ef
ps -u <user>	Processes of a user	ps -u cdac
ps aux --sort=-%cpu head	Top CPU consumers	ps aux --sort=-%cpu head
ps aux --sort=-%mem head	Top memory consumers	ps aux --sort=-%mem head
ps -p <PID> -o pid,ppid,cmd,%mem,%cpu	Custom columns for one process	ps -p 1234 -o pid,ppid,cmd
pstree -p	Process tree with PIDs	pstree -p

Command / Syntax	What it does	Example / Implementation
pgrep <name>	PIDs matching a name	pgrep -l nginx
top / htop	Live monitors	htop
kill <PID>	Send SIGTERM (polite stop)	kill 1234
kill -9 <PID>	SIGKILL: force kill (last resort)	kill -9 1234
kill -l	List all signals	kill -l
kill -HUP <PID>	Reload configuration	kill -HUP 1234
pkill <name>	Kill by name pattern	pkill firefox
killall <name>	Kill all processes with exact name	killall python3
jobs	Shell background jobs	jobs
command &	Run in background	python app.py &
Ctrl+Z	Suspend foreground job	(press Ctrl+Z)
fg %1	Bring job 1 to foreground	fg %1
bg %1	Resume stopped job in background	bg %1
nohup command &	Keep running after terminal closes	nohup python app.py > app.log 2>&1 &
disown %1	Detach a job from the shell	disown %1
nice -n 10 <cmd>	Start with lower priority (-20 high ... 19 low)	nice -n 10 ./backup.sh
renice -n 5 -p <PID>	Change priority of running process	renice -n 5 -p 1234
lsof -i :80	Which process uses port 80	sudo lsof -i :80
lsof -p <PID>	Files opened by a process	lsof -p 1234
lsof <file>	Which processes use a file	lsof /var/log/syslog
fuser -v <file/port>	Processes using a file/socket	fuser -v 22/tcp
watch -n 2 <cmd>	Re-run command every 2 seconds	watch -n 2 df -h
timeout 10 <cmd>	Stop command after 10 seconds	timeout 10 ping 8.8.8.8
strace -p <PID>	Trace system calls (debugging)	sudo strace -p 1234
time <cmd>	Measure runtime	time ./script.sh
screen / tmux	Persistent terminal sessions (detach/attach)	tmux new -s work ; tmux attach -t work

- Common signals: 1 HUP (reload), 2 INT (Ctrl+C), 9 KILL (cannot be caught), 15 TERM (default), 18 CONT, 19 STOP, 20 TSTP (Ctrl+Z).
- Process states in ps: R running, S sleeping, D uninterruptible I/O, T stopped, Z zombie.

22. Services & systemd

Command / Syntax	What it does	Example / Implementation
systemctl status <svc>	Check service state	sudo systemctl status ssh
systemctl start <svc>	Start now	sudo systemctl start ssh
systemctl stop <svc>	Stop	sudo systemctl stop ssh
systemctl restart <svc>	Stop then start	sudo systemctl restart ssh
systemctl reload <svc>	Reload config without full restart	sudo systemctl reload nginx
systemctl enable <svc>	Auto-start at boot	sudo systemctl enable ssh
systemctl enable --now <svc>	Enable and start immediately	sudo systemctl enable --now ssh
systemctl disable <svc>	Disable auto-start	sudo systemctl disable ssh
systemctl is-active <svc>	Prints active / inactive	systemctl is-active ssh
systemctl is-enabled <svc>	Prints enabled / disabled	systemctl is-enabled ssh
systemctl mask <svc> / unmask	Prevent a service from starting at all	sudo systemctl mask cups
systemctl list-units --type=service	Running services	systemctl list-units --type=service
systemctl list-unit-files	All installed units and states	systemctl list-unit-files grep enabled
systemctl --failed	Units that failed	systemctl --failed
systemctl daemon-reload	Reload unit files after editing	sudo systemctl daemon-reload
systemctl get-default	Default boot target	systemctl get-default
systemctl set-default multi-user.target	Boot to text mode (graphical.target = GUI)	sudo systemctl set-default multi-user.target
systemctl list-timers	Scheduled systemd timers	systemctl list-timers

Command / Syntax	What it does	Example / Implementation
<code>systemd-analyze blame</code>	Which units slow down boot	<code>systemd-analyze blame</code>
<code>service <svc> status</code>	Older SysV-style command	<code>sudo service ssh status</code>

Minimal custom service: `/etc/systemd/system/myapp.service`

```
[Unit]
Description=My App
After=network.target

[Service]
User=cdac
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- Then: `sudo systemctl daemon-reload && sudo systemctl enable --now myapp`

23. Logs & Journals

Command / Syntax	What it does	Example / Implementation
<code>/var/log/syslog</code> (Debian) or <code>/var/log/messages</code> (RHEL)	General system log	<code>tail -f /var/log/syslog</code>
<code>/var/log/auth.log</code> (Debian) or <code>/var/log/secure</code> (RHEL)	Authentication / sudo / SSH logs	<code>sudo grep 'Failed' /var/log/auth.log</code>
<code>/var/log/dmesg</code> , <code>/var/log/kern.log</code>	Kernel and boot messages	
<code>/var/log/apt/</code> , <code>/var/log/dpkg.log</code>	Package install history	
<code>journalctl</code>	All journal logs	<code>journalctl</code>
<code>journalctl -u <svc></code>	Logs of one service	<code>journalctl -u ssh</code>
<code>journalctl -f</code>	Follow live	<code>journalctl -fu nginx</code>
<code>journalctl -b</code>	Current boot (-b -1 = previous boot)	<code>journalctl -b -1</code>
<code>journalctl -p err</code>	Only errors and worse	<code>journalctl -p err -b</code>
<code>journalctl --since '1 hour ago'</code>	Time filter (also --until)	<code>journalctl --since today</code>
<code>journalctl --disk-usage</code>	Journal size	<code>journalctl --disk-usage</code>
<code>dmesg</code>	Kernel ring buffer	<code>dmesg tail</code>
<code>dmesg -T --level=err,warn</code>	Errors with readable time	<code>dmesg -T --level=err,warn</code>
<code>logger 'message'</code>	Write a message to the system log	<code>logger 'Backup finished'</code>
<code>logrotate -d /etc/logrotate.conf</code>	Test log rotation (debug)	<code>sudo logrotate -d /etc/logrotate.conf</code>

24. Scheduling Jobs (cron, at, timers)

Command / Syntax	What it does	Example / Implementation
<code>crontab -e</code>	Edit your cron table	<code>crontab -e</code>
<code>crontab -l</code>	List your cron jobs	<code>crontab -l</code>
<code>crontab -r</code>	Remove all your cron jobs	<code>crontab -r</code>
<code>sudo crontab -u <user> -l</code>	View another user's cron jobs	<code>sudo crontab -u cdac -l</code>
<code>/etc/crontab</code> , <code>/etc/cron.d/</code>	System-wide cron files (has extra user column)	
<code>/etc/cron.daily hourly weekly monthly</code>	Drop scripts to run on those schedules	
<code>at 10:30 tomorrow</code>	One-time job (end with Ctrl+D)	<code>echo 'bash backup.sh' at 02:00</code>
<code>atq / atrm <id></code>	List / delete pending at jobs	<code>atq</code>
<code>systemctl list-timers</code>	systemd timers	<code>systemctl list-timers --all</code>
<code>sleep 5</code>	Pause (s, m, h, d suffixes)	<code>sleep 5m</code>

- Cron format: minute hour day-of-month month day-of-week command (fields: 0-59, 0-23, 1-31, 1-12, 0-7 with 0/7 = Sunday).

```
* /5 * * * * /home/cdac/check.sh          # every 5 minutes
0 2 * * *   /home/cdac/backup.sh          # daily at 02:00
```

```

30 18 * * 1-5 /home/cdac/report.sh      # weekdays at 18:30
0 0 1 * * /home/cdac/monthly.sh        # 1st of each month at midnight
@reboot /home/cdac/start.sh            # at every boot
0 2 * * * /home/cdac/backup.sh >> /home/cdac/backup.log 2>&1 # log output

```

- Always use absolute paths in cron; cron has a minimal PATH and environment.

25. Networking

Command / Syntax	What it does	Example / Implementation
ip a / ip addr show	Interfaces, MAC and IP addresses	ip a
ip -br a	Brief interface/IP table	ip -br a
ip link	Link-layer info; add 'set eth0 up/down' to toggle	sudo ip link set eth0 up
sudo ip addr add 192.168.1.50/24 dev eth0	Add temporary IP address	sudo ip addr add 192.168.1.50/24 dev eth0
ip route	Routing table / default gateway	ip route
sudo ip route add default via <gw>	Add default route	sudo ip route add default via 192.168.1.1
ip -s link	Interface packet statistics	ip -s link
ifconfig	Legacy interface tool (net-tools)	ifconfig
netstat -r	Legacy routing table	netstat -r
ss -tulnp	Listening TCP/UDP ports + processes	sudo ss -tulnp
ss -ta / ss -ua	All TCP / UDP sockets	ss -ta
ss -s	Socket summary	ss -s
ss -tn state established	Only established TCP connections	ss -tn state established
cat /etc/hosts	Local hostname mappings	cat /etc/hosts
cat /etc/resolv.conf	DNS servers in use	cat /etc/resolv.conf
resolvectl status	systemd-resolved DNS status	resolvectl status
ip neigh / arp -a	Neighbour (ARP) cache	ip neigh
ping -c 4 <host>	Test connectivity/latency	ping -c 4 8.8.8.8
traceroute <host>	Trace packet path	traceroute google.com
tracepath <host>	Alternative path tracing	tracepath google.com
mtr <host>	Live traceroute + ping (if installed)	mtr google.com
dig <domain>	DNS query	dig example.com
dig +short <domain> / dig MX <domain>	Short answer / specific record type	dig +short example.com
nslookup <domain>	Simpler DNS lookup	nslookup example.com
host <domain>	Quick DNS lookup	host example.com
whois <domain>	Domain registration info	whois example.com
curl <url>	Fetch a URL (print to screen)	curl https://example.com
curl -I <url>	Headers only	curl -I https://example.com
curl -O <url>	Download keeping file name (-o name to rename)	curl -O https://site/file.zip
curl -X POST -d 'a=1' <url>	POST request	curl -X POST -d 'a=1' https://httpbin.org/post
wget <url>	Download file (-c resume, -r recursive)	wget -c https://site/big.iso
nc -zv <host> <port>	Test whether a port is open	nc -zv 10.0.2.15 22
telnet <host> <port>	Basic TCP test	telnet example.com 80
nmap <host>	Port scan (only on systems you own/are authorised to test)	nmap -sV 192.168.1.10
tcpdump -i eth0 port 80	Capture packets (root)	sudo tcpdump -i eth0 -c 10 port 80
ethtool eth0	NIC speed/duplex/link	sudo ethtool eth0
nmcli device status	NetworkManager device list	nmcli device status
nmcli connection show	Saved connections	nmcli connection show
nmcli dev wifi list / connect	Wi-Fi scan / connect	nmcli dev wifi connect MySSID password 'pass'

Command / Syntax	What it does	Example / Implementation
<code>sudo systemctl restart NetworkManager</code>	Restart networking (Ubuntu desktop/RHEL)	<code>sudo systemctl restart NetworkManager</code>
<code>sudo netplan apply</code>	Apply <code>/etc/netplan/*.yaml</code> (Ubuntu server)	<code>sudo netplan apply</code>

Firewall

Command / Syntax	What it does	Example / Implementation
<code>sudo ufw status verbose</code>	Ubuntu firewall status	<code>sudo ufw status verbose</code>
<code>sudo ufw enable / disable</code>	Turn firewall on / off	<code>sudo ufw enable</code>
<code>sudo ufw allow 22/tcp</code>	Allow SSH (do this BEFORE enabling over SSH)	<code>sudo ufw allow 22/tcp</code>
<code>sudo ufw deny <port> / delete allow <port></code>	Block / remove rule	<code>sudo ufw delete allow 80/tcp</code>
<code>sudo firewall-cmd --list-all</code>	RHEL/Fedora firewalld status	<code>sudo firewall-cmd --list-all</code>
<code>sudo firewall-cmd --add-service=http --permanent</code>	Open a service permanently	<code>sudo firewall-cmd --reload</code>
<code>sudo iptables -L -n -v</code>	List legacy iptables rules	<code>sudo iptables -L -n -v</code>
<code>sudo nft list ruleset</code>	List nftables rules	<code>sudo nft list ruleset</code>

26. SSH - Remote Login

- SSH securely connects to a remote machine, runs commands and supports secure file transfer (default port 22).
- Ubuntu client: `sudo apt update && sudo apt install openssh-client`
- Ubuntu server: `sudo apt update && sudo apt install openssh-server`
- Start/enable: `sudo systemctl enable --now ssh` | Check: `sudo systemctl status ssh` (service is called `sshd` on RHEL)

Command / Syntax	What it does	Example / Implementation
<code>ssh <user>@<host></code>	Connect to a remote host	<code>ssh cdac@10.0.2.15</code>
<code>ssh -p 2222 <user>@<host></code>	Custom SSH port	<code>ssh -p 2222 cdac@10.0.2.15</code>
<code>ssh <user>@<host> '<cmd>'</code>	Run one remote command	<code>ssh cdac@10.0.2.15 'df -h'</code>
<code>ssh -i ~/.ssh/key <user>@<host></code>	Use a specific private key	<code>ssh -i ~/.ssh/id_rsa cdac@host</code>
<code>ssh -v <user>@<host></code>	Verbose output for debugging (-vv, -vvv more)	<code>ssh -v cdac@host</code>
<code>ssh -X <user>@<host></code>	Forward GUI applications	<code>ssh -X cdac@host</code>
<code>ssh-keygen -t ed25519 -C "comment"</code>	Generate a key pair (<code>~/.ssh/id_ed25519</code>)	<code>ssh-keygen -t ed25519 -C "cdac laptop"</code>
<code>ssh-copy-id <user>@<host></code>	Install your public key for password-less login	<code>ssh-copy-id cdac@10.0.2.15</code>
<code>ssh-keygen -R <host></code>	Remove old host key from <code>known_hosts</code>	<code>ssh-keygen -R 10.0.2.15</code>
<code>ssh-add ~/.ssh/id_ed25519</code>	Load key into <code>ssh-agent</code>	<code>eval "\$(ssh-agent -s)" && ssh-add</code>
<code>ssh -L 8080:localhost:80 <user>@<host></code>	Local port forward (tunnel)	<code>ssh -L 8080:localhost:80 cdac@host</code>
<code>ssh -D 1080 <user>@<host></code>	SOCKS proxy through the server	<code>ssh -D 1080 cdac@host</code>
<code>sftp <user>@<host></code>	Interactive secure file transfer (get, put, ls, cd, exit)	<code>sftp cdac@10.0.2.15</code>
<code>~/.ssh/config</code>	Per-host shortcuts (Host, HostName, User, Port, IdentityFile)	Host web / HostName 10.0.2.15 / User cdac
<code>~/.ssh/authorized_keys</code>	Public keys allowed to log in (chmod 600)	<code>chmod 700 ~/.ssh ; chmod 600 ~/.ssh/authorized_keys</code>
<code>/etc/ssh/sshd_config</code>	Server config: Port, PermitRootLogin no, PasswordAuthentication no	<code>sudo systemctl restart ssh</code>

27. SCP - Secure File Copy

Command / Syntax	What it does	Example / Implementation
<code>scp file user@host:/path/</code>	Local -> remote	<code>scp report.txt cdac@10.0.2.15:/home/cdac/</code>
<code>scp user@host:/path/file ./</code>	Remote -> local	<code>scp cdac@10.0.2.15:/tmp/log.txt ./</code>
<code>scp -r dir user@host:/path/</code>	Copy directory recursively	<code>scp -r project/ cdac@10.0.2.15:/home/cdac/</code>
<code>scp -p file user@host:/path/</code>	Preserve file attributes	<code>scp -p script.sh cdac@10.0.2.15:/tmp/</code>

Command / Syntax	What it does	Example / Implementation
scp -P 2222 file user@host:/path/	Custom SSH port (capital P)	scp -P 2222 a.txt cdac@host:/tmp/
scp -i ~/.ssh/id_rsa file user@host:/path/	Use specific key	scp -i ~/.ssh/id_rsa a.txt cdac@host:/tmp/
scp -C file user@host:/path/	Compress during transfer	scp -C big.log cdac@host:/tmp/
scp u1@h1:/f u2@h2:/d	Remote to remote copy	scp cdac@h1:/tmp/a.txt cdac@h2:/tmp/

28. rsync - Synchronization & Backup

Command / Syntax	What it does	Example / Implementation
rsync src/ dest/	Local synchronization (trailing / on src = copy contents)	rsync db_backup/ backup/
rsync -av src/ dest/	Archive + verbose sync	rsync -av project/ backup/project/
rsync -avz src/ user@host:/path/	Remote sync with compression	rsync -avz db_backup/ cdac@host:/home/cdac/db_backup/
rsync -av --delete src/ dest/	Mirror: delete extra files in destination	rsync -av --delete data/ backup/data/
rsync -av -e ssh src/ user@host:/path/	Sync over SSH	rsync -av -e ssh src/ cdac@host:/home/cdac/src/
rsync -av -e 'ssh -p 2222' src/ host:/p/	Custom SSH port	rsync -av -e 'ssh -p 2222' src/ cdac@host:/tmp/
rsync -avn src/ dest/	Dry run - show what would happen (-n)	rsync -avn --delete src/ dest/
rsync -av --progress src/ dest/	Show progress	rsync -av --progress big/ dest/
rsync -av --exclude '*.log' src/ dest/	Skip patterns	rsync -av --exclude 'node_modules' app/ backup/
rsync -avP src/ dest/	Progress + resume partial transfers	rsync -avP big.iso host:/tmp/

29. Archives & Compression (tar, gzip, zip)

tar

Command / Syntax	What it does	Example / Implementation
tar -cvf a.tar dir/	Create archive (c create, v verbose, f file)	tar -cvf backup.tar project/
tar -czvf a.tar.gz dir/	Create gzip-compressed archive	tar -czvf backup.tar.gz project/
tar -cjvf a.tar.bz2 dir/	Create bzip2 archive	tar -cjvf backup.tar.bz2 project/
tar -cJvf a.tar.xz dir/	Create xz archive (best compression)	tar -cJvf backup.tar.xz project/
tar -xvf a.tar	Extract (x extract); format auto-detected	tar -xvf backup.tar.gz
tar -xzvf a.tar.gz -C /dir	Extract into another directory	tar -xzvf backup.tar.gz -C /tmp/restore
tar -tvf a.tar.gz	List contents without extracting	tar -tvf backup.tar.gz
tar -xzvf a.tar.gz file	Extract a single file	tar -xzvf backup.tar.gz project/a.txt
tar --exclude='*.log' -czf a.tar.gz dir/	Exclude patterns	tar --exclude='*.log' -czf app.tar.gz app/

gzip / bzip2 / xz

Command / Syntax	What it does	Example / Implementation
gzip file / gunzip file.gz	Compress / decompress single file (replaces original)	gzip app.log
gzip -k file	Keep original	gzip -k app.log
zcat / zless file.gz	View compressed file without extracting	zcat app.log.gz head
bzip2 file / bunzip2 file.bz2	bzip2 compress / decompress	bzip2 data.csv
xz file / unxz file.xz	xz compress / decompress	xz data.csv

zip

Command / Syntax	What it does	Example / Implementation
zip file.zip file	Compress a file	zip report.zip report.pdf
zip files.zip a b c	Compress multiple files	zip docs.zip a.txt b.txt
zip -r project.zip project/	Compress directory recursively	zip -r project.zip project/

Command / Syntax	What it does	Example / Implementation
zip -9 archive.zip files	Maximum compression	zip -9 backup.zip *.log
zip -r archive.zip dir -x '*.log'	Exclude patterns	zip -r app.zip app/ -x '*.log'
zip -re secure.zip dir	Encrypted ZIP (asks password)	zip -re secure.zip project/
unzip archive.zip	Extract archive	unzip project.zip
unzip archive.zip -d dir	Extract to directory	unzip project.zip -d extracted/
unzip -l archive.zip	List contents	unzip -l project.zip
unzip -t archive.zip	Test integrity	unzip -t project.zip
unzip -o archive.zip	Overwrite existing files	unzip -o project.zip

checksums

Command / Syntax	What it does	Example / Implementation
md5sum file / sha256sum file	Compute hash to verify integrity	sha256sum ubuntu.iso
sha256sum -c sums.txt	Verify files listed in a checksum file	sha256sum -c SHA256SUMS

30. Package Management

Debian / Ubuntu (apt, dpkg, snap)

Command / Syntax	What it does	Example / Implementation
sudo apt update	Refresh package lists	sudo apt update
sudo apt upgrade	Upgrade installed packages	sudo apt upgrade
sudo apt full-upgrade	Upgrade, allowing removals for dependencies	sudo apt full-upgrade
sudo apt install <pkg>	Install package (add -y to auto-confirm)	sudo apt install openssh-client
sudo apt remove <pkg>	Remove package, keep config	sudo apt remove nginx
sudo apt purge <pkg>	Remove package and config	sudo apt purge nginx
sudo apt autoremove	Remove unused dependencies	sudo apt autoremove
sudo apt clean	Clear downloaded package cache	sudo apt clean
apt search <term>	Search packages	apt search openssh
apt show <pkg>	Package details	apt show openssh-client
apt list --installed	Installed packages (--upgradable for updates)	apt list --installed grep ssh
apt policy <pkg>	Installed vs candidate version	apt policy nginx
sudo dpkg -i file.deb	Install a local .deb file	sudo dpkg -i google-chrome.deb
sudo apt install -f	Fix broken dependencies	sudo apt install -f
dpkg -l grep <pkg>	List installed packages	dpkg -l grep nginx
dpkg -L <pkg>	Files installed by a package	dpkg -L nginx
dpkg -S /path/file	Which package owns a file	dpkg -S /bin/ls
sudo snap install <pkg>	Install snap package (snap list / remove)	sudo snap install vlc
sudo add-apt-repository ppa:<name>	Add a PPA repository	sudo add-apt-repository ppa:deadsnakes/ppa

RHEL / CentOS / Fedora / Rocky (dnf, yum, rpm)

Command / Syntax	What it does	Example / Implementation
sudo dnf install <pkg>	Install (yum is the older name, same syntax)	sudo dnf install httpd
sudo dnf remove <pkg>	Remove package	sudo dnf remove httpd
sudo dnf update	Update all packages	sudo dnf update
dnf search <term> / dnf info <pkg>	Search / details	dnf search nginx
dnf provides */file	Which package provides a file	dnf provides */httpd
rpm -qa grep <pkg>	List installed RPMs	rpm -qa grep httpd
rpm -ivh file.rpm	Install local RPM	sudo rpm -ivh pkg.rpm

Others

Command / Syntax	What it does	Example / Implementation
<code>sudo pacman -Syu</code>	Arch: sync and upgrade (-S install, -R remove, -Ss search)	<code>sudo pacman -S vim</code>
<code>sudo zypper install <pkg></code>	openSUSE install (zypper refresh, update)	<code>sudo zypper install nginx</code>
<code>flatpak install <app></code>	Flatpak apps	<code>flatpak install flathub org.gimp.GIMP</code>
<code>pip install <pkg> / pip3 list</code>	Python packages (prefer venv: python3 -m venv env)	<code>python3 -m venv env && source env/bin/activate</code>

31. Shell Scripting - Core Structure

- A shell script is a text file of commands executed as one unit. First line (shebang) selects the interpreter:

```
#!/bin/bash
```

Command / Syntax	What it does	Example / Implementation
<code>chmod +x script.sh</code>	Make script executable	<code>chmod +x hello.sh</code>
<code>./script.sh</code>	Run executable script	<code>./hello.sh</code>
<code>bash script.sh</code>	Run through Bash (no x bit needed)	<code>bash hello.sh</code>
<code>source script.sh / . script.sh</code>	Run in the current shell (variables persist)	<code>source ~/.bashrc</code>
<code>echo</code>	Print output	<code>echo 'Hello World'</code>
<code>read</code>	Read user input	<code>read name</code>
<code>\$(command)</code>	Command substitution	<code>today=\$(date)</code>
<code>\$((expression))</code>	Arithmetic expansion	<code>sum=\$((a+b))</code>
<code>exit 0 / exit 1</code>	End script with success / failure status	<code>exit 1</code>
<code>bash -n script.sh</code>	Syntax check without running	<code>bash -n hello.sh</code>
<code>bash -x script.sh</code>	Debug: trace every command	<code>bash -x hello.sh</code>
<code>shellcheck script.sh</code>	Static analysis for scripts (if installed)	<code>shellcheck hello.sh</code>

Template

```
#!/bin/bash
# Description: what this script does
set -euo pipefail          # stop on errors, undefined variables, pipe failures

name="${1:-World}"
echo "Hello, $name"
```

32. Bash Variables & Environment

Command / Syntax	What it does	Example / Implementation
<code>name='Aryan'</code>	Create variable (NO spaces around =)	<code>name='Aryan'</code>
<code>echo "\$name"</code>	Use variable (always quote)	<code>echo "\$name"</code>
<code>\${name}</code>	Delimited form	<code>echo "\${name}_file"</code>
<code>unset name</code>	Delete variable	<code>unset name</code>
<code>readonly PI=3.14</code>	Constant variable	<code>readonly PI=3.14</code>
<code>export VAR=value</code>	Export to child processes (environment variable)	<code>export MY_APP_NAME='LinuxGuide'</code>
<code>env</code>	Show environment variables	<code>env head</code>
<code>printenv VAR</code>	Print one variable	<code>printenv HOME</code>
<code>set</code>	All shell variables and functions	<code>set less</code>
<code>echo \$HOME</code>	Home directory	<code>echo \$HOME</code>
<code>echo \$USER</code>	Current user	<code>echo \$USER</code>
<code>echo \$SHELL</code>	Current shell	<code>echo \$SHELL</code>
<code>echo \$PATH</code>	Command search path (colon-separated)	<code>echo \$PATH</code>
<code>echo \$PWD / \$OLDPWD</code>	Current / previous directory	<code>echo \$PWD</code>
<code>echo \$LANG</code>	Locale/language	<code>echo \$LANG</code>
<code>echo \$HOSTNAME</code>	Host name	<code>echo \$HOSTNAME</code>
<code>echo \$RANDOM</code>	Random number 0-32767	<code>echo \$((RANDOM % 100))</code>

Command / Syntax	What it does	Example / Implementation
<code>export PATH="\$PATH:\$HOME/bin"</code>	Add a directory to PATH	<code>export PATH="\$PATH:\$HOME/bin"</code>
<code>~/.bashrc</code>	Runs for each interactive shell: aliases, PATH, functions	<code>echo "alias ll='ls -lah'" >> ~/.bashrc</code>
<code>~/.bash_profile</code> or <code>~/.profile</code>	Runs at login	<code>source ~/.profile</code>
<code>source ~/.bashrc</code>	Reload config without logging out	<code>source ~/.bashrc</code>
<code>/etc/environment</code> , <code>/etc/profile</code>	System-wide variables / login scripts	

33. Bash Special Variables

Command / Syntax	What it does	Example / Implementation
<code>\$0</code>	Script name	<code>echo "\$0"</code>
<code>\$1</code> , <code>\$2</code> , ...	Positional arguments (<code>\${10}</code> for more than 9)	<code>echo "\$1"</code>
<code>\$#</code>	Number of arguments	<code>echo "\$#"</code>
<code>\$@</code>	All arguments, each preserved separately	<code>for x in "\$@"; do echo "\$x"; done</code>
<code>\$*</code>	All arguments as one combined string	<code>echo "\$@"</code>
<code>\$\$</code>	PID of current shell/script	<code>echo "PID: \$\$"</code>
<code>\$!</code>	PID of last background job	<code>sleep 10 & echo \$!</code>
<code>\$?</code>	Exit status of previous command (0 = success)	<code>ls /nope; echo "\$?"</code>
<code>\${1:-default}</code>	Use default if unset or empty	<code>name=\${1:-Anonymous}</code>
<code>\${var:?message}</code>	Abort with message if unset	<code>: "\${DB:?DB is required}"</code>
<code>shift</code>	Drop \$1 and move the others down	<code>shift</code>

34. Bash Input, Tests & Conditions

Command / Syntax	What it does	Example / Implementation
<code>read name</code>	Basic input	<code>read name; echo "Hello \$name"</code>
<code>read -p 'Prompt: ' x</code>	Input with prompt	<code>read -p 'Age: ' age</code>
<code>read -s -p 'Password: ' p</code>	Hidden input	<code>read -s -p 'Password: ' password</code>
<code>read -t 5 x</code>	Input with timeout	<code>read -t 5 response</code>
<code>read -a arr</code>	Read words into an array	<code>read -a items</code>

File tests

Command / Syntax	What it does	Example / Implementation
<code>[-f file]</code>	Regular file exists	<code>if [-f "\$file"]; then echo yes; fi</code>
<code>[-d dir]</code>	Directory exists	<code>if [-d "\$dir"]; then echo yes; fi</code>
<code>[-e path]</code>	Path exists	<code>if [-e "\$path"]; then echo yes; fi</code>
<code>[-r file]</code> / <code>[-w file]</code> / <code>[-x file]</code>	Readable / writable / executable	<code>if [-x script.sh]; then ./script.sh; fi</code>
<code>[-s file]</code>	Exists and not empty	<code>if [-s "\$file"]; then echo yes; fi</code>
<code>[-L file]</code>	Is a symbolic link	<code>[-L /usr/bin/python]</code>
<code>[file1 -nt file2]</code>	file1 newer than file2	<code>[a.txt -nt b.txt]</code>

String tests

Command / Syntax	What it does	Example / Implementation
<code>[-z "\$s"]</code> / <code>[-n "\$s"]</code>	String empty / not empty	<code>if [-z "\$name"]; then echo empty; fi</code>
<code>["\$a" = "\$b"]</code> / <code>!=</code>	Strings equal / not equal	<code>["\$ans" = "y"]</code>
<code>[[\$s == pat*]]</code>	Pattern matching (Bash <code>[[]]</code>)	<code>[[\$file == *.txt]]</code>
<code>[[\$s =~ regex]]</code>	Regex match	<code>[[\$email =~ ^[a-z]+@[a-z]+\.\com\$]]</code>

Number tests

Command / Syntax	What it does	Example / Implementation
-eq / -ne	Equal / not equal	["\$a" -eq 5]
-lt / -le	Less than / less or equal	["\$a" -lt 10]
-gt / -ge	Greater than / greater or equal	["\$a" -ge 18]
((a > b))	Arithmetic comparison	if ((a % 2 == 0)); then echo even; fi
!	Negate a test	if [! -d "\$dir"]; then mkdir "\$dir"; fi

35. if / elif / else, AND / OR

Pattern:

```
if [ condition ]; then
    command
elif [ condition ]; then
    command
else
    command
fi
```

- AND: [cond1] && [cond2] or [cond1 -a cond2] or [[c1 && c2]]
- OR: [cond1] || [cond2] or [[c1 || c2]]
- Example: if ["\$age" -ge 18] && ["\$license" = "y"]; then echo 'Can drive'; fi
- Test a command directly: if grep -q 'ERROR' app.log; then echo found; fi (-q = quiet, only exit status)

case statement

```
case "$choice" in
    1) echo 'View files' ;;
    2) echo 'View processes' ;;
    3) echo 'View disk usage' ;;
    start|START|Start) echo 'Starting' ;;
    *.txt) echo 'Text file' ;;
    *) echo 'Invalid option' ;;
esac
```

- Useful for menus and pattern matching (*.txt, *.sh, *.jpg) and multiple alternatives with |.

36. Loops

Command / Syntax	What it does	Example / Implementation
for x in list; do ...; done	Repeat for each item	for f in *.txt; do echo "\$f"; done
for ((i=1;i<=5;i++)); do ...; done	C-style counting loop	for ((i=1;i<=5;i++)); do echo \$i; done
for i in {1..10}; do ...; done	Range loop	for i in {1..10..2}; do echo \$i; done
while condition; do ...; done	Repeat while true	while [\$i -le 10]; do echo \$i; i=\$((i+1)); done
until condition; do ...; done	Repeat until true	until [\$i -gt 10]; do echo \$i; i=\$((i+1)); done
while true; do ...; done	Infinite loop (exit with break)	while true; do date; sleep 1; done
break	Exit loop	if ["\$x" = stop]; then break; fi
continue	Skip current iteration	if ["\$x" = skip]; then continue; fi
select opt in A B Quit; do ...; done	Auto-numbered menu	select o in start stop quit; do echo "\$o"; break; done

37. Reading Files in Bash

```
while read -r line; do
    echo "Processing: $line"
done < sample.txt

# split fields on a delimiter
while IFS=: read -r user x uid rest; do
    echo "$user has UID $uid"
done < /etc/passwd
```

- Other useful forms: head -n 3 file, cat -n file, readarray -t lines < file (lines go into array \${lines[@]}).
- Always use read -r (do not mangle backslashes) and quote variables.

38. Functions

```
function greet() {
    local name="$1"
    echo "Hello, $name"
    return 0
}
greet "Aryan"
result=$(greet "Aryan") # capture function output
```

- local creates a function-local variable; otherwise variables are global.
- Functions receive arguments as \$1, \$2... (\$# and \$@ work too); return gives an exit status 0-255, echo gives text output.
- Functions make scripts reusable and easier to maintain; put shared ones in a file and load with source lib.sh.

39. Arrays, Strings & Arithmetic in Bash

Command / Syntax	What it does	Example / Implementation
arr=(a b c)	Create indexed array	fruits=(apple banana cherry)
\${arr[0]}	One element	echo "\${fruits[0]}"
\${arr[@]}	All elements	for f in "\${fruits[@]}"; do echo "\$f"; done
\${#arr[@]}	Number of elements	echo "\${#fruits[@]}"
arr+=(d)	Append element	fruits+=(mango)
declare -A map	Associative array (Bash 4+)	declare -A age; age[alice]=30; echo "\${age[alice]}"
\${#str}	String length	echo "\${#name}"
\${str:2:3}	Substring (offset 2, length 3)	echo "\${name:0:3}"
\${str/old/new}	Replace first / \${str//old/new} all	echo "\${path//\\/\\/}_"
\${str#pre} / \${str##pre}	Strip shortest / longest prefix	f=a.b.c; echo "\${f#*.}"
\${str%suf} / \${str%%suf}	Strip shortest / longest suffix	echo "\${f%.txt}"
\${str^^} / \${str,,}	Uppercase / lowercase	echo "\${name^^}"
\$((a + b))	Integer math: + - * / % **	echo \$((7 % 3))
let x=x+1 / ((x++))	Increment variable	((count++))
bc <<< "scale=2; 10/3"	Floating-point math	echo "scale=2; 10/3" bc
expr 5 + 3	Older arithmetic helper	expr 5 + 3

40. Script Robustness & Error Handling

Command / Syntax	What it does	Example / Implementation
set -e	Exit immediately if a command fails	set -e
set -u	Error on undefined variable	set -u
set -o pipefail	Pipeline fails if any stage fails	set -o pipefail
set -x	Print each command before running (debug)	set -x
trap 'cmd' EXIT	Run cleanup when script exits	trap 'rm -f "\$tmp"' EXIT
trap 'echo Interrupted; exit 1' INT	Handle Ctrl+C	trap 'echo bye' INT TERM
mktemp / mktemp -d	Create safe temp file / directory	tmp=\$(mktemp)
command -v tool >/dev/null	Check that a tool exists	command -v git >/dev/null { echo 'git missing'; exit 1; }
["\$EUID" -ne 0] && echo 'run as root'	Check for root	if ["\$EUID" -ne 0]; then echo 'Use sudo'; exit 1; fi
\$?	Check last command result	cp a b echo 'copy failed'

getopts - parse options

```
while getopts "f:vh" opt; do
    case $opt in
        f) file="$OPTARG" ;;
        v) verbose=1 ;;
        h) echo "Usage: $0 [-f file] [-v]"; exit 0 ;;
        *) exit 1 ;;
    esac
done
```

Here-document

```
cat << 'EOF' > config.txt
name=demo
path=$HOME      # quoted EOF keeps $HOME literal
EOF
```

41. Date & Time

Command / Syntax	What it does	Example / Implementation
date	Current date/time	date
date +%Y-%m-%d	Formatted date	date +%Y-%m-%d
date +%H:%M:%S	Formatted time	date +%H:%M:%S
date +%Y%m%d_%H%M%S	Timestamp for file names	cp a.txt a_\$(date +%Y%m%d_%H%M%S).txt
date -d 'tomorrow' / '2 days ago'	Relative dates	date -d '+7 days' +%F
date +%s	Unix epoch seconds	date +%s
date -d @1700000000	Epoch to date	date -d @1700000000
cal / cal -3	Calendar	cal -3
timedatectl	Clock and timezone status	timedatectl
sudo hwclock --show	Hardware clock	sudo hwclock --show

- Format codes: %Y year, %m month, %d day, %H hour, %M minute, %S second, %A weekday name, %F = %Y-%m-%d, %T = %H:%M:%S.

42. Kernel, Boot & Power

Command / Syntax	What it does	Example / Implementation
uname -r	Running kernel version	uname -r
lsmod	Loaded kernel modules	lsmod head
modinfo <module>	Details of a module	modinfo e1000e
sudo modprobe <module>	Load a module (-r to remove)	sudo modprobe loop
cat /proc/cmdline	Kernel boot parameters	cat /proc/cmdline
sysctl -a grep <key>	Kernel runtime parameters	sysctl net.ipv4.ip_forward
sudo sysctl -w <key>=<val>	Change parameter until reboot (persist in /etc/sysctl.conf)	sudo sysctl -w net.ipv4.ip_forward=1
/boot/grub/grub.cfg	GRUB menu (edit /etc/default/grub, then update-grub / grub2-mkconfig)	sudo update-grub
systemctl get-default	Current runlevel/target	systemctl get-default
sudo reboot	Restart	sudo reboot
sudo shutdown -h now	Power off immediately	sudo shutdown -h now
sudo shutdown -r +10 'msg'	Reboot in 10 minutes with message (shutdown -c cancels)	sudo shutdown -r +10 'Maintenance'
sudo poweroff / sudo halt	Power off	sudo poweroff
last reboot	History of reboots	last reboot head
ulimit -a	Per-user resource limits	ulimit -n

43. Security & Hardening Basics

- Principle of least privilege: give users and services only the access they need.

Command / Syntax	What it does	Example / Implementation
sudo apt update && sudo apt upgrade	Keep system patched	sudo apt update && sudo apt upgrade -y
sudo ufw enable	Enable firewall (allow SSH first)	sudo ufw allow OpenSSH && sudo ufw enable
sudo grep 'Failed password' /var/log/auth.log	Spot brute-force SSH attempts	sudo grep 'Failed password' /var/log/auth.log tail
sudo lastb head	Recent failed logins	sudo lastb head
sudo apt install fail2ban	Auto-ban IPs with repeated failures	sudo systemctl enable --now fail2ban
PermitRootLogin no	In /etc/ssh/sshd_config: disable root SSH login	sudo systemctl restart ssh

Command / Syntax	What it does	Example / Implementation
sudo chage -M 90 <user>	In sshd_config: use SSH keys only	sudo systemctl restart ssh
find / -perm -4000 2>/dev/null	Enforce password expiry	sudo chage -M 90 cdac
find / -nouser -o -nogroup 2>/dev/null	Audit SUID binaries	find / -perm -4000 -type f 2>/dev/null
find / -nouser -o -nogroup 2>/dev/null	Files with no valid owner	find / -nouser 2>/dev/null
sudo ss -tulnp	Check what is listening on the network	sudo ss -tulnp
getenforce / sestatus	SELinux status (RHEL family)	getenforce
sudo setenforce 0	SELinux permissive until reboot (for testing only)	sudo setenforce 0
sudo aa-status	AppArmor status (Ubuntu/Debian)	sudo aa-status
openssl rand -base64 24	Generate a random password/secret	openssl rand -base64 24
gpg -c file	Symmetric encrypt a file (gpg file.gpg to decrypt)	gpg -c secrets.txt
base64 file / base64 -d	Encode / decode base64 (encoding, not encryption)	echo hi base64

44. Other Useful Tools (git, containers, misc)

Command / Syntax	What it does	Example / Implementation
git init	Create repository	git init
git clone <url>	Copy a repository	git clone https://github.com/user/repo.git
git status	Show changes	git status
git add .	Stage all changes	git add .
git commit -m 'msg'	Save a commit	git commit -m 'Add feature'
git log --oneline	Short history	git log --oneline -10
git pull / git push	Fetch+merge / upload commits	git push origin main
git branch / git checkout -b 	List / create+switch branch	git checkout -b feature1
git diff	Show unstaged changes	git diff
docker ps -a	List containers (docker images lists images)	docker ps -a
docker run -it ubuntu bash	Run an interactive container	docker run -it ubuntu bash
docker exec -it <c> bash	Shell into running container	docker exec -it web bash
docker logs -f <c>	Follow container logs	docker logs -f web
tmux new -s name	New persistent terminal session (Ctrl+b d detach)	tmux attach -t name
screen -S name	Alternative multiplexer (Ctrl+a d detach, screen -r)	screen -r name
bc	Command-line calculator	echo '2^10' bc
seq 1 5	Print number sequence	seq 1 2 9
yes head -3	Repeat a string	yes hello head -3
jq '.key' file.json	Process JSON (if installed)	curl -s api jq '.name'
sleep 2	Pause	sleep 2
lsb_release -a / neofetch	System summary	neofetch
script session.log	Record a terminal session (exit to stop)	script session.log

45. Practical Mini-Implementations

Files and text

```

wc -w file.txt           # A. number of words
wc -l file.txt           # B. number of lines
find . -type f -mtime -1 # C. files modified in the last day
grep -n -i 'error' app.log # D. find errors in a log
grep -ci 'error' app.log  # E. count errors
sort file.txt | uniq -c | sort -nr # F. frequency count, most common first
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head # G. top IPs in a web log
sed -i.bak 's/old/new/g' *.conf # H. bulk replace with backups

```

System checks

```
ps aux --sort=-%cpu | head      # top CPU consumers
ps aux --sort=-%mem | head     # top memory consumers
sudo ss -tulnp                  # listening ports
df -h ; du -sh * | sort -h     # disk usage and largest directory
free -h ; uptime               # memory and load
journalctl -p err -b           # errors since boot
```

Backup and logging

```
zip -r backup_$(date +%Y%m%d).zip project/
tar -czf backup_$(date +%F).tar.gz project/
echo "$(date): Backup completed" >> backup.log
rsync -av --delete project/ /backup/project/
```

Users, groups and permissions

```
sudo useradd -m -s /bin/bash alice && sudo passwd alice
sudo groupadd devs && sudo usermod -aG devs alice
sudo mkdir /srv/team && sudo chgrp devs /srv/team && sudo chmod 2770 /srv/team
sudo chown -R alice:devs project/ && chmod -R u=rwX,g=rwX,o= project/
```

Scripts

```
# Even numbers 1-20
for ((i=1; i<=20; i++)); do
    if (( i % 2 == 0 )); then echo "$i"; fi
done

# Read a file and print each line
while read -r line; do echo "$line"; done < input.txt

# Simple menu
echo '1. Files'; echo '2. Processes'; echo '3. Disk'
read -p 'Choice: ' c
case $c in
    1) ls -la ;;
    2) ps aux ;;
    3) df -h ;;
    *) echo 'Invalid choice' ;;
esac

# Disk space alert
usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$usage" -gt 80 ]; then echo "Disk usage high: ${usage}%"; fi

# Bulk-create users from a file (one name per line)
while read -r u; do sudo useradd -m "$u" && echo "created $u"; done < users.txt
```

46. Troubleshooting Playbook

Command / Syntax	What it does	Example / Implementation
Command not found	Check spelling, PATH and install state	<code>which cmd ; echo \$PATH ; apt search cmd</code>
Permission denied	Check owner, mode, directory x bit, ACLs, SELinux	<code>ls -l file ; namei -l /full/path ; id</code>
Disk full	Find usage, then big files and inode exhaustion	<code>df -h ; df -i ; du -h --max-depth=1 sort -h</code>
System slow	Load, CPU, RAM, swap, I/O	<code>uptime ; top ; free -h ; iostat -x 1 3</code>
Service will not start	Status, logs, config test	<code>systemctl status svc ; journalctl -xeu svc</code>
Port already in use	Find the process holding the port	<code>sudo ss -tulnp grep :80 ; sudo lsof -i :80</code>
No network	Link, IP, gateway, DNS, firewall	<code>ip a ; ip route ; ping 8.8.8.8 ; ping google.com</code>
Cannot SSH	Service up? port open? firewall? key permissions?	<code>ssh -v user@host ; nc -zv host 22 ; chmod 600 ~/.ssh/id_*</code>
User cannot access shared dir	Group membership, relogin, directory permissions	<code>id user ; ls -ld dir ; getfacl dir</code>
Script will not run	Executable bit, shebang, Windows line endings	<code>chmod +x s.sh ; head -1 s.sh ; dos2unix s.sh</code>
Process is hung	Find and terminate gently, then forcefully	<code>kill PID ; sleep 5 ; kill -9 PID</code>
Unexpected reboot or crash	Previous boot's logs	<code>journalctl -b -1 -p err ; last reboot</code>

Command / Syntax	What it does	Example / Implementation
wrong time	Check timezone and NTP	timedatectl ; sudo timedatectl set-ntp true

47. High-Value Command Differences

Command / Syntax	What it does	Example / Implementation
find vs grep	find locates files by attributes (name, type, size, time); grep searches text inside files	find . -name '*.log' vs grep -r 'ERROR' .
find vs locate	find scans live filesystem; locate uses a database (fast but may be stale)	find /home -name report.txt vs locate report.txt
cp vs mv	cp copies and keeps the source; mv moves/renames and removes the old location	cp a.txt b.txt vs mv a.txt b.txt
mv vs rename	mv works on single paths; rename does bulk pattern-based changes	mv old.txt new.txt vs rename 's/\.txt\$/\.md/' *.txt
scp vs rsync	scp is simple secure copy; rsync syncs and transfers only differences	scp -r dir host:/tmp/ vs rsync -avz dir/ host:/tmp/
chmod vs chown	chmod changes permissions; chown changes ownership	chmod 750 file vs chown user:group file
chgrp vs chown	chgrp changes only the group; chown can change user and/or group	chgrp dev file vs chown user:dev file
useradd vs adduser	useradd is low-level (needs -m for home); adduser is the interactive Debian wrapper	useradd -m u vs adduser u
groupadd vs addgroup	groupadd is the standard tool; addgroup is the Debian-style wrapper	groupadd dev vs addgroup dev
usermod -G vs -aG	-G replaces supplementary groups; -aG appends	usermod -aG dev u (safe) vs usermod -G dev u
su vs sudo	su switches user (needs that user's password); sudo runs one command with your own password and logging	su - root vs sudo cmd
hard link vs symlink	hard link = same inode; symlink = pointer to a path	ln a b vs ln -s a b
> vs >>	> overwrites; >> appends	echo hi > f vs echo hi >> f
kill vs kill -9	SIGTERM lets a process clean up; SIGKILL cannot be caught	kill PID vs kill -9 PID
apt vs apt-get vs dpkg	apt = friendly front end; apt-get = scripting-stable; dpkg = low-level .deb tool	apt install x vs dpkg -i x.deb
systemctl enable vs start	enable = at boot; start = now; use enable --now for both	systemctl enable --now ssh
ip a vs ifconfig	ip is modern; ifconfig is legacy	ip a
ss vs netstat	ss is modern socket tool; netstat is legacy	ss -tulnp
tar vs zip	tar bundles (compress with gz/xz) and preserves Linux permissions/owners; zip is cross-platform	tar -czf a.tgz d/ vs zip -r a.zip d/
cron vs at	cron repeats on a schedule; at runs once	crontab -e vs at 02:00

48. Quick Revision Cheat Sheet

Command / Syntax	What it does	Example / Implementation
Navigation	pwd ls -la cd dir cd .. cd ~ cd -	
Files	touch f cat f cp f d/ mv f g rm f ln -s t l	
Directories	mkdir d mkdir -p a/b rm -r d rmdir d tree	
Search	find . -name '*.txt' grep -rin 'error' . locate file	
Text	head tail -f wc -l sort uniq -c cut sed awk diff	
Permissions	ls -l chmod 750 f chown user:group f umask chmod g+s dir getfacl/setfacl	

Command / Syntax	What it does	Example / Implementation
Users	whoami id who w last useradd usermod userdel passwd	
Groups	groups groupadd groupmod groupdel usermod -aG gpasswd -a/-d newgrp getent group	
Privileges	sudo su - visudo sudo -l	
Processes	ps aux top kill PID pkill jobs fg bg nohup nice lsof	
Services	systemctl start/stop/restart/enable/status journalctl -u svc	
Scheduling	crontab -e crontab -l at systemctl list-timers	
System	uname -a hostnamectl lscpu free -h df -h uptime dmesg	
Storage	lsblk fdisk -l blkid du -sh mount umount mkfs fsck	
Network	ip a ip route ss -tulnp ping dig curl ufw	
Remote	ssh user@host ssh-keygen ssh-copy-id scp rsync -avz	
Archive	tar -czf a.tgz dir tar -xzf a.tgz zip -r a.zip dir unzip a.zip	
Packages	apt update/upgrade/install/remove dnf install dpkg -i snap install	
Shell	echo read if case for while function trap set -euo pipefail	
Debug	bash -x shellcheck echo "\$?" journalctl -xe strace	

49. Safety & Good Practice

- Run `pwd` and `ls` before destructive operations so you know where you are.
- Prefer `rm -i` when learning; be especially careful with `rm -rf` (never run it with an unchecked variable, e.g. `rm -rf "$DIR/"` when `DIR` is empty).
- Use `sudo` only when required. Avoid running normal work as root.
- Preview bulk renames with `rename -n`, and `rsync` with `--dry-run (-n)`, before applying.
- Check permissions with `ls -l` before changing them recursively; avoid `chmod 777` - use groups, ACLs or `setgid` directories instead.
- Always add `-a` when using `usermod -G` (use `usermod -aG`) so existing groups are not wiped.
- Group membership changes only take effect after the user logs in again.
- Quote variables in scripts ("`$file`") to handle spaces safely; start scripts with `set -euo pipefail` where appropriate.
- Back up configuration files before editing (`cp file file.bak`) and validate before restarting services (`visudo`, `sshd -t`, `nginx -t`).
- Keep a second SSH session open when changing `sshd_config` or firewall rules so you cannot lock yourself out.
- Use key-based SSH, disable root login and password logins where possible, and keep the system updated.
- Verify the target hostname/IP and the `dd/mkfs` target device before running destructive commands.
- Combine `grep`, `awk`, `cut`, `sort` and `uniq` for log analysis rather than reading large files manually.
- Test new scripts on sample data first; never paste commands from untrusted sources without reading them.